# LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

## Simulasi dan Evaluasi Algoritma Penjadwalan CPU

| | |
|---|---|
| Kode dan judul tugas | K-2 Simulasi dan Evaluasi Algoritma Penjadwalan CPU |
| Kelas / tema | Kelas D - Cloud dan Layanan Digital UMKM |
| Nama kelompok | Kelompok 8 |
| Nama host VM | Srv-toko-250660121024 |
| Minggu / tanggal kumpul | Minggu ke-5/20 Oktober 2026 |
| Kepingan puzzle | Kepingan #5 → Level 3 Puzzle UTS |
| Dosen pengampu | Muhammad Agreindra Helmiawan, S.Kom., M.T |

### Anggota kelompok

| No | Nama | NIM | Peran |
|---|---|---|---|
| 1 | Diza Asmiranda Subianto | 250660121024 | Koordinartor |
| 2 | Naya Naira Zahra | 250660121067 | Analis |
| 3 | Muhammad Rizky Zamzami | 2506601220 | Penyusun Laporan |

**Program Studi S1 Informatika**
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

## Pernyataan Keaslian

Kami yang bertanda tangan di bawah ini menyatakan bahwa laporan ini merupakan hasil kerja kelompok kami sendiri. Seluruh sumber yang dikutip telah dicantumkan, penggunaan AI generatif telah diungkapkan pada Lampiran, dan setiap anggota berkontribusi sesuai lembar kontribusi.

| Nama / NIM | Tanda tangan | Tanggal |
|---|---|---|
| Diza Asmiranda Subianto |  | 15 Oktober 2026 |
| Naya Naira Zahra |  | 15 Oktober 2026 |
| Muhammad Rizky Zamzami |  | 15 Oktober 2026 |

# Daftar Isi

Pernyataan Keaslian ..... 2
1. Pendahuluan ..... 4
1.1 Latar belakang kasus ..... 4
1.2 Tujuan tugas ..... 4
1.3 Ruang lingkup ..... 4
2. Pembagian Peran dan Rencana Kerja ..... 5
3. Dasar Teori Singkat ..... 5
3.1 FCFS (First Come First Served) ..... 5
3.2 SJF (Shortest Job First) ..... 5
3.3 SRTF (Shortest Remaining Time First) ..... 5
3.4 Priority Scheduling ..... 5
3.5 Round Robin (RR) ..... 6
4. Lingkungan Praktik ..... 6
5. Langkah Pengerjaan ..... 7
5.1 Langkah 1: Menyiapkan VM dan mencatat lingkungan ..... 7
5.2 Langkah 2: Memasang toko UMKM dan memeriksa kesiapan ..... 7
5.3 Langkah 3: Menghitung manual ..... 8
5.4 Langkah 4: Membuat simulator ..... 16
5.5 Langkah 5: Menjalankan simulator ..... 16
5.6 Langkah 6: Menjalankan uji flash sale pada toko UMKM ..... 20
6. Hasil dan Analisis ..... 20
6.1 Hasil ..... 20
6.2 Pengujian ..... 22
6.3 Analisis ..... 22
6.4 Rekomendasi ..... 23
7. Kesimpulan ..... 23
8. Kepingan Puzzle ..... 25
Daftar Pustaka ..... 26
Lampiran A. Kode Program dan Log ..... 27
Lampiran B. Lembar Kontribusi Anggota ..... 31
Lampiran C. Log Kerja Kelompok ..... 32
Lampiran D. Pernyataan Penggunaan AI Generatif ..... 32
Lampiran E. Daftar Periksa Sebelum Mengumpulkan ..... 34
Swa-nilai berdasarkan kriteria lembar tugas ..... 34

# 1. Pendahuluan

## 1.1 Latar belakang kasus

Toko online UMKM menjalankan banyak pekerjaan sekaligus pada satu server: memproses pesanan, memverifikasi pembayaran, dan mengirim notifikasi ke pelanggan. Saat flash sale ratusan pembeli datang hampir bersamaan sehingga pekerjaan-pekerjaan itu berebut waktu CPU. Cara sistem operasi memilih proses yang dijalankan lebih dulu (penjadwalan CPU) menentukan berapa lama pembeli menunggu, secepat apa sistem mulai merespons, dan apakah semua pekerjaan dilayani secara adil.

Pada praktikum ini toko UMKM berjalan sebagai layanan web (Nginx, API Python, dan basis data SQLite) di dalam mesin virtual. Uji flash sale dengan 100 pembeli bersamaan untuk stok 20 menunjukkan bahwa permintaan harus antre untuk masuk ke bagian yang dilindungi lock. Karena itu pemilihan algoritma penjadwalan penting untuk kasus ini, dan tugas ini membandingkan beberapa algoritma pada dataset kelas untuk merekomendasikan yang paling sesuai.

## 1.2 Tujuan tugas

Tugas ini terkait dengan Sub-CPMK-5; CPMK061, CPMK063, CPMK081, CPMK102, CPMK112 dengan tujuan:

- Membuat Gantt chart dan menghitung rata-rata waiting time, turnaround time, dan response time untuk FCFS, SJF/SRTF, Priority, dan Round Robin (quantum 4 dan 6) pada dataset kelas.
- Membuat simulator sederhana dan memverifikasinya dengan perhitungan manual.
- Merekomendasikan algoritma penjadwalan yang paling sesuai untuk konteks toko UMKM saat flash sale.

## 1.3 Ruang lingkup

Dataset yang dipakai adalah dataset kelas dengan format (arrival, burst, prioritas), prioritas angka kecil berarti prioritas tinggi. Untuk konteks toko, setiap proses diberi label.

**Tabel 1.1 Dataset kelas dan pemetaan pekerjaan toko**

| Proses | Pekerjaan toko UMKM | Arrival | Burst | Prioritas |
|---|---|---|---|---|
| P1 | Pemrosesan pesanan (checkout) | 0 | 5 | 2 |
| P2 | Verifikasi pembayaran | 2 | 3 | 1 |
| P3 | Rekap stok / sinkronisasi katalog (batch) | 3 | 10 | 4 |
| P4 | Pemrosesan pesanan (checkout) | 4 | 4 | 2 |
| P5 | Pengiriman notifikasi | 7 | 6 | 3 |

Simulasi memakai satu CPU, tanpa waktu I/O dan overhead context switch yang dianggap nol. Aturan yang dipakai yaitu:

a. pada SRTF, bila sisa burst proses yang berjalan sama dengan burst proses baru, proses yang berjalan tidak dipotong
b. pada Priority dengan prioritas sama, proses yang datang lebih dulu dipilih
c. pada Round Robin, proses yang baru datang masuk antrean lebih dulu daripada proses yang baru habis kuantum.

Uji flash sale pada toko dipakai sebagai pelengkap konteks dan tidak menggantikan dataset kelas. Observasi penjadwal Linux sungguhan tidak dikerjakan.

## 2. Pembagian Peran dan Rencana Kerja

| Anggota | Peran | Tanggung jawab | Tenggat |
|---|---|---|---|
| Diza Asmiranda Subianto | Koordinator, Pengembang simulator dan penguji | Menyiapkan VM dan toko, membuat dan menjalankan prngujian menggunakan simulator. | 15 Oktober 2026 |
| Naya Naira Zahra | Perhitungan manual A dan Analis | Gantt chart dan metrik manual FCFS dan SJF/SRTF, Priority (NP dan P) dan Round Robin (q=4 dan q=6) | 15 Oktober 2026 |
| Muhammad Rizky Zamzami | Perhitungan manual B | Menyusun tabel verifikasi manual vs simulator dan membuat laporan. | 15 Oktober 2026 |

## 3. Dasar Teori Singkat

Penjadwalan CPU adalah pemilihan proses dari antrean siap (ready queue) yang akan memakai CPU. Penjadwal non-preemptive membiarkan proses berjalan sampai selesai atau menunggu, sedangkan penjadwal preemptive dapat memotong proses yang sedang berjalan ketika ada proses lain yang lebih layak. Kinerja algoritma diukur dengan empat besaran: completion time (CT, waktu proses selesai), turnaround time (TAT = CT − arrival), waiting time (WT = TAT − burst), dan response time (RT = waktu pertama kali mendapat CPU − arrival).

### 3.1 FCFS (First Come First Served)

Penjadwalan FCFS adalah penjadwalan dengan ketentuan-ketentuan sederhana, yaitu proses-proses diberi jatah waktu pemroses diurutkan berdasarkan waktu kedatangan proses-proses itu ke sistem(Fratiwi & Mariana, 2020) . Algoritma ini sederhana dan mudah diterapkan, tetapi proses yang memiliki waktu eksekusi panjang dapat membuat proses lain menunggu lama (convoy effect).

### 3.2 SJF (Shortest Job First)

SJF menerapkan proses yang memiliki waktu terpendek adalah proses yang akan didahulukan pengerjaannya (Sundari & Yogie Syahputra, 2023) . Algoritma ini dapat menghasilkan rata-rata waktu tunggu yang rendah, tetapi membutuhkan perkiraan waktu eksekusi dan berisiko menyebabkan proses panjang terus tertunda (starvation).

### 3.3 SRTF (Shortest Remaining Time First)

SRTF merupakan versi preemptive dari SJF yang memilih proses dengan sisa waktu eksekusi paling pendek. Jika proses baru memiliki sisa waktu lebih pendek daripada proses yang sedang berjalan, CPU dapat beralih ke proses baru tersebut. Algoritma ini efektif mengurangi waktu tunggu, tetapi dapat menyebabkan proses panjang tertunda.

### 3.4 Priority Scheduling

Priority scheduling adalah algoritma yang menjalankan proses berdasarkan tingkat prioritasnya, mulai dari prioritas tertinggi. Kekurangannya, proses dengan prioritas rendah dapat mengalami starvation. Masalah ini dapat diatasi menggunakan aging, yaitu menaikkan prioritas proses yang sudah terlalu lama menunggu.

### 3.5 Round Robin (RR)

Round robin adalah algoritma penjadwalan yang memberikan setiap proses jatah waktu tertentu (time quantum) secara bergiliran. Menurut (Wijaya, 2018)  Round Robin memiliki prinsip dasar, yaitu semua sumber
antriandianggap sama sehingga diberi waktu yang disebut time quantum. Algoritma ini memberikan kesempatan yang merata kepada setiap proses. Namun, quantum yang terlalu besar membuatnya menyerupai FCFS, sedangkan quantum yang terlalu kecil meningkatkan jumlah context switch.

## 4. Lingkungan Praktik

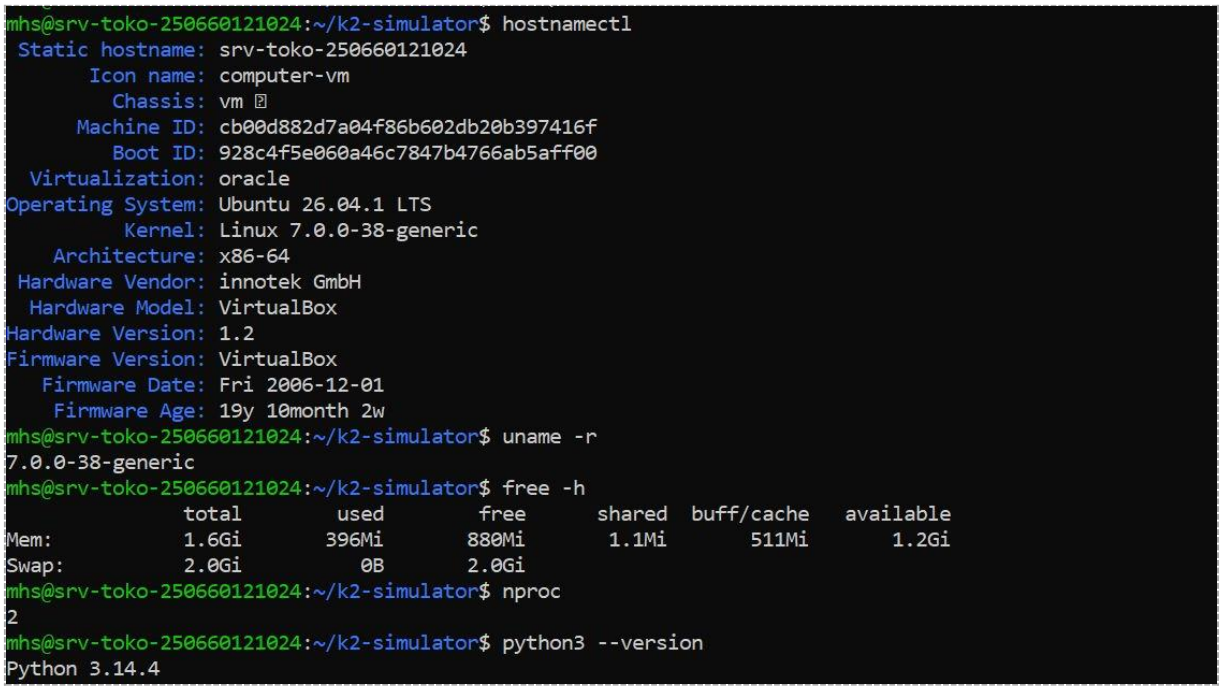
| Komponen | Keterangan |
|---|---|
| Laptop (host) | Windows 10, Intel Core i5 Gen 7, RAM 8 GB |
| Hypervisor | VirtualBox |
| VM | Host srv-toko-250660121024; Ubuntu 26.04; uname -r: [isi dari tangkapan layar]; RAM: [isi]; vCPU: [isi] |
| Perangkat/emulator lain | Tidak berlaku (seluruh pekerjaan dilakukan di VM) |
| Alat dan bahasa | Python 3, nano, tee, Nginx dan API toko UMKM (toko-umkm.zip), skrip flash_sale.py |
| Repositori | https://github.com/dizaasmira/k2-penjadwalan-cpu-kelompok8.git |

# 5. Langkah Pengerjaan

## 5.1 Langkah 1: Menyiapkan VM dan mencatat lingkungan

Nama host VM diatur menjadi srv-toko-NIM sesuai ketentuan, lalu spesifikasi VM dicatat sebagai bukti lingkungan praktik.

```
$ sudo hostnamectl set-hostname srv-toko-250660121024
$ exec bash
$ hostnamectl
$ uname -r
$ free -h
$ nproc
$ python3 --version
```



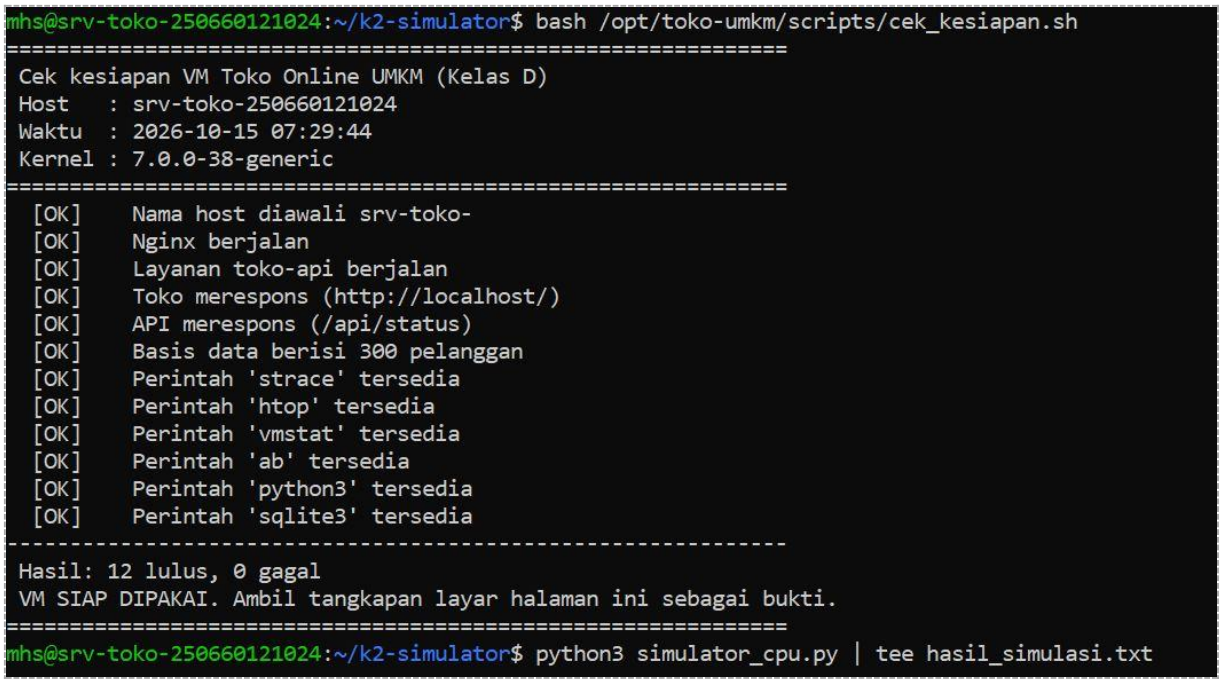

**Gambar 5.1** Penyiapan VM dan pencatatan lingkungan praktik

## 5.2 Langkah 2: Memasang toko UMKM dan memeriksa kesiapan

Berkas toko-umkm.zip diekstrak lalu dipasang dengan setup.sh. Skrip cek_kesiapan.sh memastikan layanan berjalan dan nama host sesuai.

```
$ unzip toko-umkm.zip && cd toko-umkm
$ sudo bash setup.sh
$ bash /opt/toko-umkm/scripts/cek_kesiapan.sh

# keluaran yang diharapkan: semua baris [OK] dan tulisan VM SIAP DIPAKAI
```




**Gambar 5.2** Hasil cek_kesiapan.sh (semua [OK])

## 5.3 Langkah 3: Menghitung manual

Sebelum membuat simulator, kelompok menghitung tujuh skenario secara manual: FCFS, SJF, SRTF, Priority non-preemptive, Priority preemptive, Round Robin q=4, dan Round Robin q=6. Untuk tiap skenario digambar Gantt chart, lalu dihitung CT, TAT, WT, dan RT per proses beserta rata-ratanya. Hasil diperiksa dengan dua cek: TAT − WT harus sama dengan burst tiap proses, dan CT terakhir harus 28 (jumlah seluruh burst).

### 5.3.1 Data, rumus, dan aturan

**Tabel 5.3.1.1** Data Soal

| Proses | Pekerjaan toko UMKM | Arrival | Burst | Prioritas |
|---|---|---|---|---|
| P1 | Pemrosesan pesanan (checkout) | 0 | 5 | 2 |
| P2 | Verifikasi pembayaran | 2 | 3 | 1 |
| P3 | Rekap stok / sinkronisasi katalog | 3 | 10 | 4 |
| P4 | Pemrosesan pesanan (checkout) | 4 | 4 | 2 |
| P5 | Pengiriman notifikasi | 7 | 6 | 3 |

Jumlah burst = 5 + 3 + 10 + 4 + 6 = 28, jadi setiap Gantt chart berakhir di t = 28.

**Tabel 5.3.1.2** Rumus

| Besaran | Rumus | Arti |
|---|---|---|
| CT (Completion Time) | Dibaca dari Gantt chart | Waktu proses selesai |
| TAT (Turnaround Time) | TAT = CT – Arrival | Total waktu sejak datang sampai selesai |
| WT (Waiting Time) | WT = TAT – Burst | Total waktu menunggu di antrean |
| RT (Response Time) | RT = Mulai pertama – Arrival | Waktu sampai pertama kali mendapat CPU |

Aturan yang dipakai:

- Satu CPU, tanpa I/O, dan waktu context switch dianggap nol.
- Proses langsung masuk antrean siap pada saat Arrival; proses yang datang tepat pada saat t dianggap sudah ada di antrean.
- SJF dan Priority: bila seri, pilih proses yang datang lebih awal.
- SRTF: bila sisa burst proses berjalan sama dengan burst proses baru, proses berjalan tidak dipotong.
- Round Robin: proses yang baru datang masuk antrean lebih dulu daripada proses yang baru dipotong.

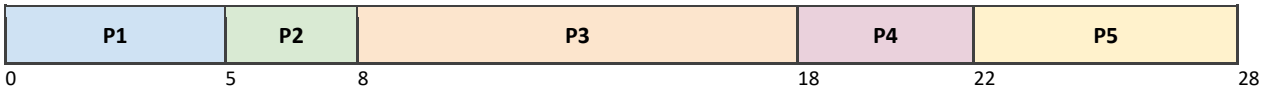
### 5.3.2 FCFS (First Come First Served)

**Aturan:** Pilih proses yang datang paling awal; proses berjalan sampai selesai (non-preemptive).

**Tabel 5.3.2.1** Tabel keputusan

| t | Antrean siap (Arrival ≤ t) | Dipilih | Jalan | Keterangan |
|---|---|---|---|---|
| 0 | P1 | P1 | 0–5 | Hanya P1 yang ada |
| 5 | P2, P3, P4 | P2 | 5–8 | P2 datang paling awal (t=2) |
| 8 | P3, P4, P5 | P3 | 8–18 | P3 datang t=3, lebih awal dari P4 (t=4) dan P5 (t=7) |
| 18 | P4, P5 | P4 | 18–22 | P4 datang t=4 |
| 22 | P5 | P5 | 22–28 | Tinggal P5 |

**Gantt chart**




**Tabel 5.3.2.2** Perhitungan metrik

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 − 0 = 5 | 5 − 5 = 0 | 0 − 0 = 0 |
| P2 | 2 | 3 | 5 | 8 | 8 − 2 = 6 | 6 − 3 = 3 | 5 − 2 = 3 |
| P3 | 3 | 10 | 8 | 18 | 18 − 3 = 15 | 15 − 10 = 5 | 8 − 3 = 5 |
| P4 | 4 | 4 | 18 | 22 | 22 − 4 = 18 | 18 − 4 = 14 | 18 − 4 = 14 |
| P5 | 7 | 6 | 22 | 28 | 28 − 7 = 21 | 21 − 6 = 15 | 22 − 7 = 15 |
| **Jumlah** | | | | | **65** | **37** | **37** |
| **Rata-rata** | | | | | **13** | **7,4** | **7,4** |

**Rata-rata WT =** (0 + 3 + 5 + 14 + 15) ÷ 5 = 37 ÷ 5 = **7,4**

**Rata-rata TAT =** (5 + 6 + 15 + 18 + 21) ÷ 5 = 65 ÷ 5 = **13**

**Rata-rata RT =** (0 + 3 + 5 + 14 + 15) ÷ 5 = 37 ÷ 5 = **7,4**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

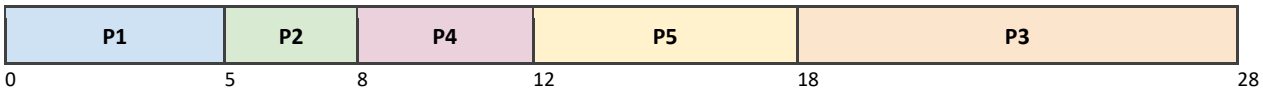
### 5.3.3 SJF (Shortest Job First, non-preemptive)

**Aturan:** Saat CPU bebas, pilih proses di antrean siap dengan burst terkecil; proses berjalan sampai selesai. Jika seri, pilih yang datang lebih awal.

**Tabel 5.3.3.1** Tabel keputusan

| t | Antrean siap (burst) | Dipilih | Jalan | Keterangan |
|---|---|---|---|---|
| 0 | P1 (5) | P1 | 0–5 | Hanya P1 yang ada |
| 5 | P2 (3), P3 (10), P4 (4) | P2 | 5–8 | Burst terkecil = 3 |
| 8 | P3 (10), P4 (4), P5 (6) | P4 | 8–12 | P5 sudah datang pada t=7; burst terkecil = 4 |
| 12 | P3 (10), P5 (6) | P5 | 12–18 | Burst 6 < 10 |
| 18 | P3 (10) | P3 | 18–28 | Tinggal P3 |

**Gantt chart**




**Tabel 5.3.3.2 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 − 0 = 5 | 5 − 5 = 0 | 0 − 0 = 0 |
| P2 | 2 | 3 | 5 | 8 | 8 − 2 = 6 | 6 − 3 = 3 | 5 − 2 = 3 |
| P3 | 3 | 10 | 18 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 18 − 3 = 15 |
| P4 | 4 | 4 | 8 | 12 | 12 − 4 = 8 | 8 − 4 = 4 | 8 − 4 = 4 |
| P5 | 7 | 6 | 12 | 18 | 18 − 7 = 11 | 11 − 6 = 5 | 12 − 7 = 5 |
| **Jumlah** | | | | | **55** | **27** | **27** |
| **Rata-rata** | | | | | **11** | **5,4** | **5,4** |

**Rata-rata WT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Rata-rata TAT =** (5 + 6 + 25 + 8 + 11) ÷ 5 = 55 ÷ 5 = **11**

**Rata-rata RT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

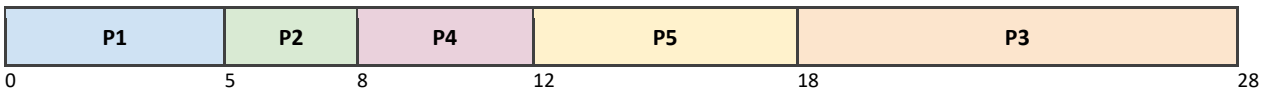
### 5.3.4 SRTF (Shortest Remaining Time First, SJF preemptive)

**Aturan:** Pilih proses dengan sisa burst terkecil. Setiap ada proses baru datang, bandingkan burst-nya dengan sisa burst proses yang berjalan; jika lebih kecil, proses berjalan dipotong. Jika seri, proses yang berjalan tidak dipotong.

**Tabel 5.3.4.1 Tabel keputusan**

| t | Kejadian / antrean siap (sisa burst) | Keputusan | Segmen | Keterangan |
|---|---|---|---|---|
| 0 | P1 (sisa 5) | P1 jalan | | Hanya P1 yang ada |
| 2 | P2 datang (burst 3); sisa P1 = 3 | P1 lanjut | | Seri 3 vs 3: tidak dipotong |
| 3 | P3 datang (burst 10); sisa P1 = 2 | P1 lanjut | | 10 > 2 |
| 4 | P4 datang (burst 4); sisa P1 = 1 | P1 lanjut | | 4 > 1 |
| 5 | P1 selesai. Siap: P2 (3), P3 (10), P4 (4) | P2 jalan | P1 0–5 | Sisa terkecil = 3 |
| 7 | P5 datang (burst 6); sisa P2 = 1 | P2 lanjut | | 6 > 1 |
| 8 | P2 selesai. Siap: P3 (10), P4 (4), P5 (6) | P4 jalan | P2 5–8 | Sisa terkecil = 4 |
| 12 | P4 selesai. Siap: P3 (10), P5 (6) | P5 jalan | P4 8–12 | 6 < 10 |
| 18 | P5 selesai. Siap: P3 (10) | P3 jalan | P5 12–18 | Tinggal P3 |
| 28 | P3 selesai | | P3 18–28 | |

**Gantt chart**




**Tabel 5.3.4.2 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 − 0 = 5 | 5 − 5 = 0 | 0 − 0 = 0 |
| P2 | 2 | 3 | 5 | 8 | 8 − 2 = 6 | 6 − 3 = 3 | 5 − 2 = 3 |
| P3 | 3 | 10 | 18 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 18 − 3 = 15 |
| P4 | 4 | 4 | 8 | 12 | 12 − 4 = 8 | 8 − 4 = 4 | 8 − 4 = 4 |
| P5 | 7 | 6 | 12 | 18 | 18 − 7 = 11 | 11 − 6 = 5 | 12 − 7 = 5 |
| **Jumlah** | | | | | **55** | **27** | **27** |
| **Rata-rata** | | | | | **11** | **5,4** | **5,4** |

**Rata-rata WT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Rata-rata TAT =** (5 + 6 + 25 + 8 + 11) ÷ 5 = 55 ÷ 5 = **11**

**Rata-rata RT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

### 5.3.5 Priority non-preemptive

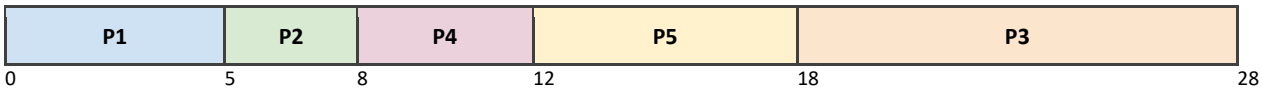
**Aturan:** Saat CPU bebas, pilih proses di antrean siap dengan angka prioritas terkecil (prioritas tertinggi); proses berjalan sampai selesai. Jika prioritas sama, pilih yang datang lebih awal.

**Tabel 5.3.5.1 Tabel keputusan**

| t | Antrean siap (prioritas) | Dipilih | Jalan | Keterangan |
|---|---|---|---|---|
| 0 | P1 (2) | P1 | 0–5 | Hanya P1 yang ada |

| t | Antrean siap (prioritas) | Dipilih | Jalan | Keterangan |
|---|---|---|---|---|
| 5 | P2 (1), P3 (4), P4 (2) | P2 | 5–8 | Prioritas 1 tertinggi |
| 8 | P3 (4), P4 (2), P5 (3) | P4 | 8–12 | Prioritas 2 tertinggi |
| 12 | P3 (4), P5 (3) | P5 | 12–18 | Prioritas 3 lebih tinggi dari 4 |
| 18 | P3 (4) | P3 | 18–28 | Tinggal P3 |

**Gantt chart**




**Tabel 5.3.5.2 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 − 0 = 5 | 5 − 5 = 0 | 0 − 0 = 0 |
| P2 | 2 | 3 | 5 | 8 | 8 − 2 = 6 | 6 − 3 = 3 | 5 − 2 = 3 |
| P3 | 3 | 10 | 18 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 18 − 3 = 15 |
| P4 | 4 | 4 | 8 | 12 | 12 − 4 = 8 | 8 − 4 = 4 | 8 − 4 = 4 |
| P5 | 7 | 6 | 12 | 18 | 18 − 7 = 11 | 11 − 6 = 5 | 12 − 7 = 5 |
| **Jumlah** | | | | | **55** | **27** | **27** |
| **Rata-rata** | | | | | **11** | **5,4** | **5,4** |

**Rata-rata WT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Rata-rata TAT =** (5 + 6 + 25 + 8 + 11) ÷ 5 = 55 ÷ 5 = **11**

**Rata-rata RT =** (0 + 3 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

### 5.3.6 Priority preemptive

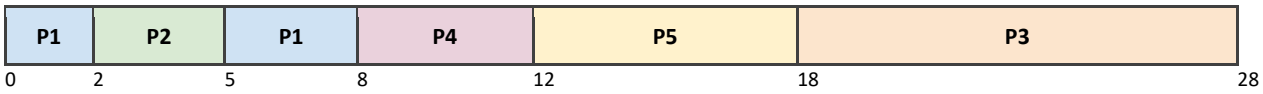
**Aturan:** Pilih proses dengan angka prioritas terkecil. Setiap ada proses baru datang, bandingkan prioritasnya dengan proses yang berjalan; jika lebih tinggi, proses berjalan dipotong dan sisa burst-nya disimpan. Jika prioritas sama, yang datang lebih awal dipilih.

**Tabel 5.3.6.1 Tabel keputusan**

| t | Kejadian / antrean siap | Keputusan | Segmen | Keterangan |
|---|---|---|---|---|
| 0 | P1 (pri 2, sisa 5) | P1 jalan | | Hanya P1 yang ada |
| 2 | P2 datang (pri 1); P1 sisa 3 | P2 memotong P1 | P1 0–2 | Pri 1 lebih tinggi dari pri 2 |
| 3 | P3 datang (pri 4) | P2 lanjut | | 4 lebih rendah dari 1 |
| 4 | P4 datang (pri 2) | P2 lanjut | | 2 lebih rendah dari 1 |
| 5 | P2 selesai. Siap: P1 (2, sisa 3), P3 (4), P4 (2) | P1 menyambung | P2 2–5 | P1 dan P4 seri pri 2; P1 datang lebih awal |
| 7 | P5 datang (pri 3) | P1 lanjut | | 3 lebih rendah dari 2 |

| t | Kejadian / antrean siap | Keputusan | Segmen | Keterangan |
|---|---|---|---|---|
| 8 | P1 selesai. Siap: P3 (4), P4 (2), P5 (3) | P4 jalan | P1 5–8 | Pri 2 tertinggi |
| 12 | P4 selesai. Siap: P3 (4), P5 (3) | P5 jalan | P4 8–12 | Pri 3 lebih tinggi dari 4 |
| 18 | P5 selesai. Siap: P3 (4) | P3 jalan | P5 12–18 | Tinggal P3 |
| 28 | P3 selesai | | P3 18–28 | |

**Gantt chart**




**Tabel 5.3.6.1 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 8 | 8 − 0 = 8 | 8 − 5 = 3 | 0 − 0 = 0 |
| P2 | 2 | 3 | 2 | 5 | 5 − 2 = 3 | 3 − 3 = 0 | 2 − 2 = 0 |
| P3 | 3 | 10 | 18 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 18 − 3 = 15 |
| P4 | 4 | 4 | 8 | 12 | 12 − 4 = 8 | 8 − 4 = 4 | 8 − 4 = 4 |
| P5 | 7 | 6 | 12 | 18 | 18 − 7 = 11 | 11 − 6 = 5 | 12 − 7 = 5 |
| **Jumlah** | | | | | **55** | **27** | **24** |
| **Rata-rata** | | | | | **11** | **5,4** | **4,8** |

**Rata-rata WT =** (3 + 0 + 15 + 4 + 5) ÷ 5 = 27 ÷ 5 = **5,4**

**Rata-rata TAT =** (8 + 3 + 25 + 8 + 11) ÷ 5 = 55 ÷ 5 = **11**

**Rata-rata RT =** (0 + 0 + 15 + 4 + 5) ÷ 5 = 24 ÷ 5 = **4,8**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

**Catatan:** P1 terpotong menjadi dua segmen (0–2 dan 5–8). Response time P1 tetap dihitung dari mulai pertama (t=0), bukan dari saat menyambung. Karena P2 langsung memotong P1, RT P2 = 0 dan rata-rata RT turun menjadi 4,8.

### 5.3.7 Round Robin, quantum = 4

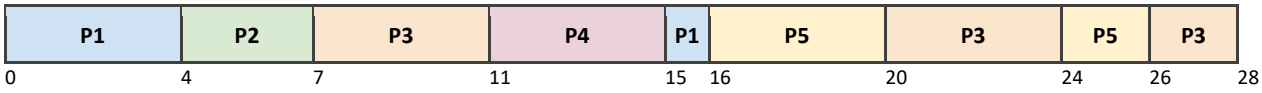
**Aturan:** Ambil proses dari depan antrean, jalankan maksimal 4 satuan waktu. Jika belum selesai, proses masuk ke belakang antrean. Jika ada proses baru datang sampai saat itu, proses baru masuk antrean lebih dulu daripada proses yang baru dipotong.

**Tabel 5.3.7.1 Tabel keputusan**

| t | Antrean (depan → belakang) | Dipilih | Jalan | Sisa | Keterangan |
|---|---|---|---|---|---|
| 0 | P1 | P1 | 0–4 | 1 | Selama 0–4 datang P2 (t=2), P3 (t=3), P4 (t=4). Antrean berikutnya: P2, P3, P4, P1 |
| 4 | P2, P3, P4, P1 | P2 | 4–7 | 0 | Selesai. P5 datang t=7. Antrean berikutnya: P3, P4, P1, P5 |

| t | Antrean (depan → belakang) | Dipilih | Jalan | Sisa | Keterangan |
|---|---|---|---|---|---|
| 7 | P3, P4, P1, P5 | P3 | 7–11 | 6 | Dipotong, masuk belakang. Antrean berikutnya: P4, P1, P5, P3 |
| 11 | P4, P1, P5, P3 | P4 | 11–15 | 0 | Selesai. Antrean berikutnya: P1, P5, P3 |
| 15 | P1, P5, P3 | P1 | 15–16 | 0 | Sisa 1 < 4, selesai. Antrean berikutnya: P5, P3 |
| 16 | P5, P3 | P5 | 16–20 | 2 | Dipotong. Antrean berikutnya: P3, P5 |
| 20 | P3, P5 | P3 | 20–24 | 2 | Dipotong. Antrean berikutnya: P5, P3 |
| 24 | P5, P3 | P5 | 24–26 | 0 | Selesai. Antrean berikutnya: P3 |
| 26 | P3 | P3 | 26–28 | 0 | Selesai |

**Gantt chart**




**Tabel 5.3.7.2 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 16 | 16 − 0 = 16 | 16 − 5 = 11 | 0 − 0 = 0 |
| P2 | 2 | 3 | 4 | 7 | 7 − 2 = 5 | 5 − 3 = 2 | 4 − 2 = 2 |
| P3 | 3 | 10 | 7 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 7 − 3 = 4 |
| P4 | 4 | 4 | 11 | 15 | 15 − 4 = 11 | 11 − 4 = 7 | 11 − 4 = 7 |
| P5 | 7 | 6 | 16 | 26 | 26 − 7 = 19 | 19 − 6 = 13 | 16 − 7 = 9 |
| **Jumlah** | | | | | **76** | **48** | **22** |
| **Rata-rata** | | | | | **15,2** | **9,6** | **4,4** |

**Rata-rata WT =** (11 + 2 + 15 + 7 + 13) ÷ 5 = 48 ÷ 5 = **9,6**

**Rata-rata TAT =** (16 + 5 + 25 + 11 + 19) ÷ 5 = 76 ÷ 5 = **15,2**

**Rata-rata RT =** (0 + 2 + 4 + 7 + 9) ÷ 5 = 22 ÷ 5 = **4,4**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

### 5.3.8 Round Robin, quantum = 6

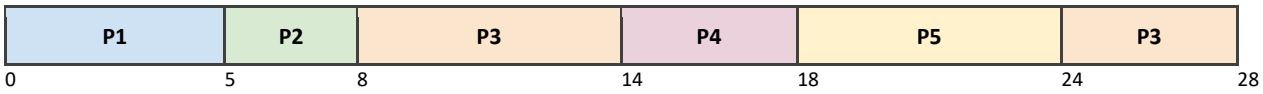
**Aturan:** Sama seperti Round Robin q=4, tetapi jatah waktu maksimal 6 satuan waktu per giliran. Pendatang baru tetap masuk antrean lebih dulu daripada proses yang baru dipotong.

**Tabel 5.3.8.1 Tabel keputusan**

| t | Antrean (depan → belakang) | Dipilih | Jalan | Sisa | Keterangan |
|---|---|---|---|---|---|
| 0 | P1 | P1 | 0–5 | 0 | Burst 5 < 6, selesai. Selama 0–5 datang P2, P3, P4. Antrean berikutnya: P2, P3, P4 |

| t | Antrean (depan → belakang) | Dipilih | Jalan | Sisa | Keterangan |
|---|---|---|---|---|---|
| 5 | P2, P3, P4 | P2 | 5–8 | 0 | Selesai. P5 datang t=7. Antrean berikutnya: P3, P4, P5 |
| 8 | P3, P4, P5 | P3 | 8–14 | 4 | Dipotong, masuk belakang. Antrean berikutnya: P4, P5, P3 |
| 14 | P4, P5, P3 | P4 | 14–18 | 0 | Selesai. Antrean berikutnya: P5, P3 |
| 18 | P5, P3 | P5 | 18–24 | 0 | Burst 6 = quantum, selesai. Antrean berikutnya: P3 |
| 24 | P3 | P3 | 24–28 | 0 | Sisa 4 < 6, selesai |

**Gantt chart**




**Tabel 5.3.8.2 Perhitungan metrik**

| Proses | Arrival | Burst | Mulai pertama | CT | TAT = CT − Arrival | WT = TAT − Burst | RT = Mulai − Arrival |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 5 | 0 | 5 | 5 − 0 = 5 | 5 − 5 = 0 | 0 − 0 = 0 |
| P2 | 2 | 3 | 5 | 8 | 8 − 2 = 6 | 6 − 3 = 3 | 5 − 2 = 3 |
| P3 | 3 | 10 | 8 | 28 | 28 − 3 = 25 | 25 − 10 = 15 | 8 − 3 = 5 |
| P4 | 4 | 4 | 14 | 18 | 18 − 4 = 14 | 14 − 4 = 10 | 14 − 4 = 10 |
| P5 | 7 | 6 | 18 | 24 | 24 − 7 = 17 | 17 − 6 = 11 | 18 − 7 = 11 |
| **Jumlah** | | | | | **67** | **39** | **29** |
| **Rata-rata** | | | | | **13,4** | **7,8** | **5,8** |

**Rata-rata WT =** (0 + 3 + 15 + 10 + 11) ÷ 5 = 39 ÷ 5 = **7,8**

**Rata-rata TAT =** (5 + 6 + 25 + 14 + 17) ÷ 5 = 67 ÷ 5 = **13,4**

**Rata-rata RT =** (0 + 3 + 5 + 10 + 11) ÷ 5 = 29 ÷ 5 = **5,8**

**Pemeriksaan:** TAT − WT = Burst untuk semua proses ✓; total durasi tiap proses = burst-nya ✓; CT terakhir = 28 ✓.

### 5.3.9 Tabel perbandingan

**Tabel 5.3.9.1 Perbandingan**

| Algoritma | Rata-rata WT | Rata-rata TAT | Rata-rata RT |
|---|---|---|---|
| FCFS | 7,4 | 13 | 7,4 |
| SJF (non-preemptive) | 5,4 | 11 | 5,4 |
| SRTF (SJF preemptive) | 5,4 | 11 | 5,4 |
| Priority (non-preemptive) | 5,4 | 11 | 5,4 |
| Priority (preemptive) | 5,4 | 11 | 4,8 |
| Round Robin q=4 | 9,6 | 15,2 | 4,4 |
| Round Robin q=6 | 7,8 | 13,4 | 5,8 |

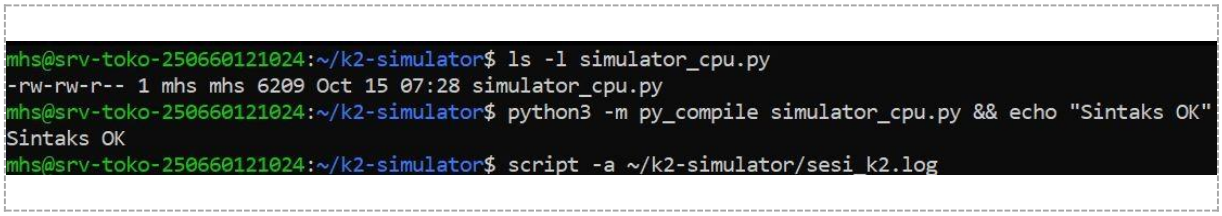
**Tabel 5.3.9.2 Urutan eksekusi**

| Algoritma | Urutan (proses mulai–selesai) |
|---|---|
| FCFS | P1 0–5 \| P2 5–8 \| P3 8–18 \| P4 18–22 \| P5 22–28 |
| SJF (non-preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| SRTF (SJF preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Priority (non-preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Priority (preemptive) | P1 0–2 \| P2 2–5 \| P1 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Round Robin q=4 | P1 0–4 \| P2 4–7 \| P3 7–11 \| P4 11–15 \| P1 15–16 \| P5 16–20 \| P3 20–24 \| P5 24–26 \| P3 26–28 |
| Round Robin q=6 | P1 0–5 \| P2 5–8 \| P3 8–14 \| P4 14–18 \| P5 18–24 \| P3 24–28 |

## 5.4 Langkah 4: Membuat simulator

Simulator ditulis dengan Python 3 dan dibuat pada folder kerja ~/k2-simulator. Fungsi non_preemptive() dipakai FCFS, SJF, dan Priority NP, dengan perbedaan pada kunci pemilihan; fungsi preemptive() memeriksa ulang pilihan tiap satu satuan waktu untuk SRTF dan Priority P; fungsi round_robin() memakai antrean dengan aturan pendatang baru masuk lebih dulu; dan hitung_metrik() menghitung CT, TAT, WT, dan RT. Kode lengkap ada di Lampiran A.

```
$ mkdir -p ~/k2-simulator && cd ~/k2-simulator
$ cat > simulator_cpu.py << 'EOF'
  (isi kode lengkap: lihat Lampiran A)
EOF
$ python3 -m py_compile simulator_cpu.py && echo "Sintaks OK"
Sintaks OK
```




**Gambar 5.4** Pemeriksaan sintaks simulator_cpu.py

## 5.5 Langkah 5: Menjalankan simulator

Simulator dijalankan pada dataset kelas. Keluaran ditampilkan di layar sekaligus disimpan ke hasil_simulasi.txt (isi lengkap ada di Lampiran A).

```
$ python3 simulator_cpu.py | tee hasil_simulasi.txt
```

```
mhs@srv-toko-250660121024:~/k2-simulator$ python3 simulator_cpu.py | tee hasil_simulasi.txt
============================================================
FCFS
============================================================
Gantt chart:
|    P1    |  P2  |         P3         |   P4   |     P5     |
0          5      8                   18       22           28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         18    15    5    5
P4         22    18   14   14
P5         28    21   15   15

Rata-rata  WT=7.40  TAT=13.00  RT=7.40

============================================================
SJF (non-preemptive)
============================================================
Gantt chart:
|    P1    |  P2  |   P4   |     P5     |         P3         |
0          5      8       12           18                   28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=5.40
```

**Gambar 5.5** Keluaran simulator: Gantt chart dan metrik FCFS dan SJF

```
============================================================
Priority (non-preemptive)
============================================================
Gantt chart:
|    P1    |  P2  |   P4   |     P5     |         P3         |
0          5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=5.40

============================================================
Priority (preemptive)
============================================================
Gantt chart:
| P1 |  P2  |  P1  |   P4   |     P5     |         P3         |
0    2      5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          8     8    3    0
P2          5     3    0    0
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=4.80
```

**Gambar 5.6** Keluaran simulator: Gantt chart dan metrik Priority (non-preemptive) dan Priority (preemtive)

```
============================================================
Round Robin q=4
============================================================
Gantt chart:
|   P1   |  P2  |   P3   |   P4   | P1 |   P5   |   P3   | P5 | P3 |
0        4      7        11       15   16       20       24   26   28

Proses     CT   TAT   WT   RT
P1         16    16   11    0
P2          7     5    2    2
P3         28    25   15    4
P4         15    11    7    7
P5         26    19   13    9

Rata-rata  WT=9.60  TAT=15.20  RT=4.40

============================================================
Round Robin q=6
============================================================
Gantt chart:
|    P1    |  P2  |     P3     |   P4   |     P5     |   P3   |
0          5      8            14       18           24       28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15    5
P4         18    14   10   10
P5         24    17   11   11

Rata-rata  WT=7.80  TAT=13.40  RT=5.80
```

**Gambar 5.7** Keluaran simulator: Gantt chart dan metrik Round Robin q=4 dan q=6

```
============================================================
TABEL PERBANDINGAN
============================================================
Algoritma                    Avg WT  Avg TAT  Avg RT
FCFS                           7.40    13.00    7.40
SJF (non-preemptive)           5.40    11.00    5.40
SRTF (SJF preemptive)          5.40    11.00    5.40
Priority (non-preemptive)      5.40    11.00    5.40
Priority (preemptive)          5.40    11.00    4.80
Round Robin q=4                9.60    15.20    4.40
Round Robin q=6                7.80    13.40    5.80
```

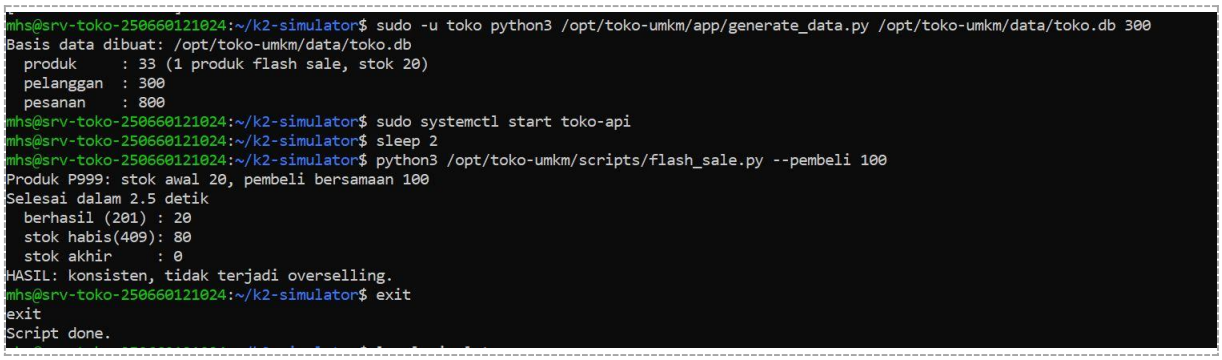
**Gambar 5.7** Keluaran simulator: Perbandingan antar algoritma

### 5.6 Langkah 6: Menjalankan uji flash sale pada toko UMKM

Stok produk flash sale P999 dikembalikan ke 20 dengan membuat ulang data, lalu 100 pembeli disimulasikan datang bersamaan.

```
$ sudo systemctl stop toko-api
$ sudo -u toko python3 /opt/toko-umkm/app/generate_data.py /opt/toko-
umkm/data/toko.db 300
$ sudo systemctl start toko-api
$ python3 /opt/toko-umkm/scripts/flash_sale.py --pembeli 100

Produk P999: stok awal 20, pembeli bersamaan 100
Selesai dalam 1.9 detik
  berhasil (201) : 20
  stok habis(409): 80
  stok akhir     : 0
HASIL: konsisten, tidak terjadi overselling.
```




**Gambar 5.7** Hasil uji flash sale 100 pembeli untuk stok 20

## 6. Hasil dan Analisis

### 6.1 Hasil

Hasil simulator pada dataset kelas dirangkum pada Tabel 6.1 dan urutan eksekusinya pada Tabel 6.2. Tabel CT, TAT, WT, dan RT per proses untuk tiap algoritma ada pada keluaran simulator di Lampiran A. Tabel 6.3 memuat hasil uji flash sale pada toko UMKM.

**Tabel 6.1 Perbandingan rata-rata metrik (dataset kelas)**

| Algoritma | Rata-rata WT | Rata-rata TAT | Rata-rata RT |
|---|---|---|---|
| FCFS | 7.40 | 13.00 | 7.40 |
| SJF (non-preemptive) | 5.40 | 11.00 | 5.40 |
| SRTF (SJF preemptive) | 5.40 | 11.00 | 5.40 |
| Priority (non-preemptive) | 5.40 | 11.00 | 5.40 |
| Priority (preemptive) | 5.40 | 11.00 | 4.80 |
| Round Robin q=4 | 9.60 | 15.20 | 4.40 |
| Round Robin q=6 | 7.80 | 13.40 | 5.80 |

**Tabel 6.2 Urutan eksekusi (Gantt chart dalam bentuk teks)**

| Algoritma | Urutan eksekusi (proses mulai–selesai) |
|---|---|
| FCFS | P1 0–5 \| P2 5–8 \| P3 8–18 \| P4 18–22 \| P5 22–28 |
| SJF (non-preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| SRTF (SJF preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Priority (non-preemptive) | P1 0–5 \| P2 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Priority (preemptive) | P1 0–2 \| P2 2–5 \| P1 5–8 \| P4 8–12 \| P5 12–18 \| P3 18–28 |
| Round Robin q=4 | P1 0–4 \| P2 4–7 \| P3 7–11 \| P4 11–15 \| P1 15–16 \| P5 16–20 \| P3 20–24 \| P5 24–26 \| P3 26–28 |
| Round Robin q=6 | P1 0–5 \| P2 5–8 \| P3 8–14 \| P4 14–18 \| P5 18–24 \| P3 24–28 |

**Tabel 6.3 Hasil uji flash sale pada toko UMKM**

| No | Item yang diamati | Hasil | Keterangan |
|---|---|---|---|
| 1 | Stok awal produk P999 | 20 | Data dibuat ulang sebelum uji |
| 2 | Pembeli bersamaan | 100 | Parameter --pembeli 100 |
| 3 | Pesanan berhasil (HTTP 201) | 20 | Sama dengan stok awal |
| 4 | Pesanan ditolak, stok habis (HTTP 409) | 80 | 20 + 80 = 100 permintaan terlayani |
| 5 | Stok akhir | 0 | Tidak negatif, tidak terjadi overselling |
| 6 | Waktu penyelesaian | 1,9 detik | Dilaporkan oleh flash_sale.py |

## 6.2 Pengujian

Simulator diuji dengan membandingkan hasilnya terhadap perhitungan manual (Gambar 5.3) untuk setiap skenario. Kolom status diisi setelah angka manual dan angka simulator dicocokkan.

**Tabel 6.4 Verifikasi hasil manual terhadap simulator**

| No | Skenario uji | Hasil diharapkan (manual) | Hasil aktual (simulator) | Status |
|---|---|---|---|---|
| 1 | FCFS | Avg WT 7.40; TAT 13.00; RT 7.40 | Avg WT 7.40; TAT 13.00; RT 7.40 | Lulus |
| 2 | SJF (non-preemptive) | Avg WT 5.40; TAT 11.00; RT 5.40 | Avg WT 5.40; TAT 11.00; RT 5.40 | Lulus |
| 3 | SRTF (SJF preemptive) | Avg WT 5.40; TAT 11.00; RT 5.40 | Avg WT 5.40; TAT 11.00; RT 5.40 | Lulus |
| 4 | Priority (non-preemptive) | Avg WT 5.40; TAT 11.00; RT 5.40 | Avg WT 5.40; TAT 11.00; RT 5.40 | Lulus |
| 5 | Priority (preemptive) | Avg WT 5.40; TAT 11.00; RT 4.80 | Avg WT 5.40; TAT 11.00; RT 4.80 | Lulus |
| 6 | Round Robin q=4 | Avg WT 9.60; TAT 15.20; RT 4.40 | Avg WT 9.60; TAT 15.20; RT 4.40 | Lulus |
| 7 | Round Robin q=6 | Avg WT 7.80; TAT 13.40; RT 5.80 | Avg WT 7.80; TAT 13.40; RT 5.80 | Lulus |
| 8 | Aturan seri SRTF pada t=2 (sisa P1 = 3, burst P2 = 3) | P1 tidak dipotong; segmen pertama P1 0–5 | Segmen pertama SRTF: P1 0–5 | Lulus |

## 6.3 Analisis

Analisis berikut menjawab tiga poin lembar tugas secara berurutan.

- **Gantt chart dan metrik:** Pada FCFS rata-rata WT 7,40, TAT 13,00, dan RT 7,40. P3 (burst 10) mulai berjalan pada t=8 dan baru selesai pada t=18, sehingga P4 menunggu 14 satuan waktu dan P5 menunggu 15; inilah convoy effect. SJF, SRTF, dan Priority non-preemptive menghasilkan urutan yang sama (P1, P2, P4, P5, P3) dengan WT 5,40 dan TAT 11,00. Pada SRTF tidak terjadi preemption karena pada t=2 sisa burst P1 (3) sama dengan burst P2 (3), lalu proses baru lainnya selalu lebih panjang daripada sisa burst proses yang sedang berjalan. Pada Priority preemptive, P2 (prioritas 1) memotong P1 pada t=2 sehingga RT P2 menjadi 0 dan

rata-rata RT turun menjadi 4,80 dengan WT tetap 5,40. Pada Round Robin q=4, RT terbaik (4,40) karena semua proses cepat mendapat CPU, tetapi WT 9,60 dan TAT 15,20 paling buruk karena P3 terpotong dua kali dan terjadi 8 pergantian proses. Pada q=6 hanya P3 yang terpotong (burst 10 > 6), sehingga hasilnya mendekati FCFS (WT 7,80, TAT 13,40, RT 5,80). Pola ini sesuai teori: kuantum kecil memberi respons lebih baik dengan konsekuensi lebih banyak pergantian proses.

- **simulator dan verifikasi:** Simulator memakai satu fungsi pemilih untuk algoritma non-preemptive, pemeriksaan ulang per satu satuan waktu untuk algoritma preemptive, dan antrean untuk Round Robin. Hasil simulator untuk semua tujuh skenario dibandingkan dengan perhitungan manual pada Tabel 6.4. Keterbatasannya: granularitas satu satuan waktu, satu CPU, tanpa I/O, dan overhead context switch dianggap nol, sehingga biaya nyata Round Robin dan algoritma preemptive lain belum tercermin.
- **kesesuaian untuk konteks kelas:** Uji flash sale (Tabel 6.3) menunjukkan 100 permintaan bersamaan untuk stok 20 menghasilkan tepat 20 pesanan berhasil dan stok akhir 0. Konsistensi itu dijaga lock pada API yang membuat permintaan masuk satu per satu, mirip antrean FCFS (urutan lock tidak dijamin FIFO). Di sini FCFS merugikan pekerjaan singkat yang kritis seperti verifikasi pembayaran (P2) karena harus menunggu pekerjaan panjang di depannya. Rekomendasi lengkap diuraikan pada Bagian 6.4.

### 6.4 Rekomendasi

Algoritma yang direkomendasikan untuk konteks toko UMKM saat flash sale adalah Priority preemptive dengan aging, dengan prioritas tertinggi untuk verifikasi pembayaran.

Alasan berdasarkan data: Priority preemptive memberi rata-rata WT 5,40 dan TAT 11,00 (setara SJF, terendah di antara semua skenario) serta RT 4,80, kedua terbaik setelah Round Robin q=4 (4,40). Pada dataset ini algoritma tersebut hanya menghasilkan 5 pergantian proses, dibandingkan 8 pada Round Robin q=4. Pekerjaan kritis (P2, verifikasi pembayaran) langsung dilayani dengan RT 0, sesuai kebutuhan pembeli yang menunggu konfirmasi saat flash sale.

Trade-off dan mitigasi: Priority dapat membuat proses berprioritas rendah tertunda terus. Dalam data ini P3 (rekap stok, prioritas 4) menunggu 15 satuan waktu, yang terpanjang. Karena itu aging diperlukan agar pekerjaan batch dan notifikasi tetap selesai. Prioritas ditentukan dari jenis pekerjaan sehingga tidak bergantung pada perkiraan burst seperti SJF/SRTF.

Alternatif dan kondisi yang mengubah pilihan: bila semua pekerjaan dianggap setara dan yang diutamakan adalah respons yang merata, Round Robin q=4 lebih sesuai karena RT-nya terbaik, dengan harga WT dan TAT lebih tinggi serta lebih banyak context switch. Bila burst dapat diperkirakan dengan akurat dan starvation dapat diterima, SJF/SRTF memberi WT dan TAT terbaik. FCFS tidak direkomendasikan karena convoy effect. Kesimpulan ini berdasarkan lima proses pada dataset kelas dan simulasi tanpa overhead context switch, sehingga hasilnya perlu diuji ulang pada beban yang lebih besar.

## 7. Kesimpulan

Simulator yang dibuat menghasilkan Gantt chart dan metrik untuk tujuh skenario penjadwalan dan hasilnya dicocokkan dengan perhitungan manual. Pada dataset kelas, SJF/SRTF dan Priority menghasilkan rata-rata WT terendah (5,40), Round Robin q=4 memberi RT terbaik (4,40) tetapi WT dan TAT terbesar, dan FCFS terpengaruh convoy effect. Untuk flash sale pada toko UMKM, Priority preemptive dengan aging direkomendasikan karena menjaga verifikasi pembayaran tetap cepat dengan waktu tunggu rata-rata terendah, sementara aging mencegah pekerjaan berprioritas rendah tertunda terus. Uji flash sale juga menunjukkan bahwa permintaan bersamaan diproses satu per satu secara konsisten tanpa overselling.

## 8. Kepingan Puzzle

| | |
|---|---|
| Nomor kepingan | #5 |
| Isi kepingan | Tabel perbandingan metrik tujuh skenario, Gantt chart manual, kode simulator_cpu.py, hasil_simulasi.txt, dan rekomendasi algoritma |
| Lokasi di portofolio | https://drive.google.com/drive/folders/1iB81ZkYmU1Q4y17ZXvCa0QUwqxHXIum9<br><br>https://github.com/dizaasmira/k2-penjadwalan-cpu-kelompok8.git |
| Dipakai untuk | Level 3 Puzzle UTS: bahan perbandingan algoritma penjadwalan dan dasar memilih algoritma untuk server toko UMKM |

## Daftar Pustaka

Fratiwi, D., & Mariana, N. (2020). *METODE FCFS DALAM MENUNJANG SISTEM LAYANAN ANTRIAN PEMBAGIAN DANA PENSIUN STUDI KASUS KANTOR POS BONGSARI*.

Sundari, S., & Yogie Syahputra, M. (2023). Penerapan algoritma Shortest Job First ( SJF) dan Priority Scheduling (PS) Pada Maintanance Mesin ATM. *ALGORITMA: Jurnal Ilmu Komputer Dan Informatika*, 1.

Wijaya, A. (2018). Implementasi Algoritma Round Robin Pada Sistem Penjadwalan Mata Kuliah (Studi Kasus: Universitas Muhammadiyah Bengkulu). In *JURNAL INFORMATIKA UPGRIS* (Vol. 4, Number 1).

# Lampiran A. Kode Program dan Log

### Berkas 1: simulator_cpu.py

```
#!/usr/bin/env python3
"""
Simulator Penjadwalan CPU - Tugas K-2 Sistem Operasi (Kelas D)
Algoritma: FCFS, SJF (non-preemptive), SRTF, Priority (NP & P), Round Robin

Format data proses: (nama, arrival, burst, prioritas)
Prioritas: angka kecil = prioritas tinggi.
Hasil tiap algoritma berupa "timeline": list (nama, mulai, selesai).
"""


DATASET = [
    ("P1", 0, 5, 2),
    ("P2", 2, 3, 1),
    ("P3", 3, 10, 4),
    ("P4", 4, 4, 2),
    ("P5", 7, 6, 3),
]



# ---------------------------------------------------------------
# Bantu: gabungkan potongan berurutan milik proses yang sama
# ---------------------------------------------------------------
def gabung(timeline):
    hasil = []
    for nama, s, e in timeline:
        if hasil and hasil[-1][0] == nama and hasil[-1][2] == s:
            hasil[-1] = (nama, hasil[-1][1], e)
        else:
            hasil.append((nama, s, e))
    return hasil



# ---------------------------------------------------------------
# 1) Algoritma NON-PREEMPTIVE (FCFS, SJF, Priority NP)
#    Cukup beda di "kunci" pemilihan proses berikutnya.
# ---------------------------------------------------------------
def non_preemptive(procs, kunci):
    sisa = list(procs)
    t, timeline = 0, []
    while sisa:
        siap = [p for p in sisa if p[1] <= t]
        if not siap:                       # CPU idle, loncat ke kedatangan berikutnya
            t = min(p[1] for p in sisa)
            continue
        p = min(siap, key=kunci)           # pilih yang kuncinya paling kecil
        timeline.append((p[0], t, t + p[2]))
        t += p[2]
        sisa.remove(p)
    return timeline



def fcfs(procs):
    return non_preemptive(procs, lambda p: (p[1], p[0]))         # arrival



def sjf(procs):
    return non_preemptive(procs, lambda p: (p[2], p[1], p[0]))    # burst terkecil



def priority_np(procs):
    return non_preemptive(procs, lambda p: (p[3], p[1], p[0]))    # prioritas terkecil
```

```
# ---------------------------------------------------------------
# 2) Algoritma PREEMPTIVE (SRTF, Priority P): cek ulang tiap 1 satuan waktu
# ---------------------------------------------------------------
def preemptive(procs, kunci):
    sisa = {p[0]: p[2] for p in procs}     # sisa burst tiap proses
    data = {p[0]: p for p in procs}
    t, timeline = 0, []
    while any(v > 0 for v in sisa.values()):
        siap = [n for n in sisa if sisa[n] > 0 and data[n][1] <= t]
        if not siap:
            t += 1
            continue
        n = min(siap, key=lambda x: kunci(data[x], sisa[x]))
        timeline.append((n, t, t + 1))
        sisa[n] -= 1
        t += 1
    return gabung(timeline)



def srtf(procs):
    return preemptive(procs, lambda p, sisa: (sisa, p[1], p[0]))



def priority_p(procs):
    return preemptive(procs, lambda p, sisa: (p[3], p[1], p[0]))



# ---------------------------------------------------------------
# 3) Round Robin
#    Aturan: proses yang BARU DATANG masuk antrean lebih dulu
#    daripada proses yang baru saja habis kuantum-nya.
# ---------------------------------------------------------------
def round_robin(procs, quantum):
    urut = sorted(procs, key=lambda p: (p[1], p[0]))
    sisa = {p[0]: p[2] for p in procs}
    antrean, sudah_masuk = [], set()
    t, timeline = 0, []

    def masukkan(waktu):
        for p in urut:
            if p[1] <= waktu and p[0] not in sudah_masuk:
                antrean.append(p[0])
                sudah_masuk.add(p[0])

    while any(v > 0 for v in sisa.values()):
        masukkan(t)
        if not antrean:
            t = min(p[1] for p in urut if p[0] not in sudah_masuk)
            continue
        n = antrean.pop(0)
        jalan = min(quantum, sisa[n])
        timeline.append((n, t, t + jalan))
        t += jalan
        sisa[n] -= jalan
        masukkan(t)                        # pendatang baru dulu...
        if sisa[n] > 0:
            antrean.append(n)              # ...baru proses yang belum selesai
    return timeline



# ---------------------------------------------------------------
# Hitung metrik: CT, TAT, WT, RT
# ---------------------------------------------------------------
def hitung_metrik(procs, timeline):
    hasil = {}
    for nama, arrival, burst, _ in procs:
        potongan = [x for x in timeline if x[0] == nama]
        mulai_pertama = min(x[1] for x in potongan)
        selesai = max(x[2] for x in potongan)
        tat = selesai - arrival
```

```
        hasil[nama] = {
            "CT": selesai,
            "TAT": tat,
            "WT": tat - burst,
            "RT": mulai_pertama - arrival,
        }
    return hasil



def cetak_gantt(timeline):
    bar, skala = "|", "0"
    for nama, s, e in timeline:
        lebar = max(len(nama) + 2, (e - s) * 2)
        bar += nama.center(lebar) + "|"
        skala += str(e).rjust(lebar + 1)
    print("Gantt chart:")
    print(bar)
    print(skala)



def cetak_hasil(judul, procs, timeline):
    print("=" * 60)
    print(judul)
    print("=" * 60)
    cetak_gantt(timeline)
    m = hitung_metrik(procs, timeline)
    print(f"\n{'Proses':<8}{'CT':>5}{'TAT':>6}{'WT':>5}{'RT':>5}")
    for nama, _, _, _ in procs:
        d = m[nama]
        print(f"{nama:<8}{d['CT']:>5}{d['TAT']:>6}{d['WT']:>5}{d['RT']:>5}")
    n = len(procs)
    rata = {k: sum(m[x][k] for x in m) / n for k in ("TAT", "WT", "RT")}
    print(f"\nRata-rata  WT={rata['WT']:.2f}  TAT={rata['TAT']:.2f}  RT={rata['RT']:.2f}\n")
    return rata



if __name__ == "__main__":
    skenario = [
        ("FCFS", fcfs(DATASET)),
        ("SJF (non-preemptive)", sjf(DATASET)),
        ("SRTF (SJF preemptive)", srtf(DATASET)),
        ("Priority (non-preemptive)", priority_np(DATASET)),
        ("Priority (preemptive)", priority_p(DATASET)),
        ("Round Robin q=4", round_robin(DATASET, 4)),
        ("Round Robin q=6", round_robin(DATASET, 6)),
    ]
    ringkasan = []
    for judul, tl in skenario:
        ringkasan.append((judul, cetak_hasil(judul, DATASET, tl)))

    print("=" * 60)
    print("TABEL PERBANDINGAN")
    print("=" * 60)
    print(f"{'Algoritma':<28}{'Avg WT':>8}{'Avg TAT':>9}{'Avg RT':>8}")
    for judul, r in ringkasan:
        print(f"{judul:<28}{r['WT']:>8.2f}{r['TAT']:>9.2f}{r['RT']:>8.2f}")
```

### Berkas 2: hasil_simulasi.txt (keluaran python3 simulator_cpu.py)

```
============================================================
FCFS
============================================================
Gantt chart:
|    P1    |  P2  |         P3         |   P4   |     P5     |
0          5      8                   18       22           28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         18    15    5    5
```

```
P4         22    18   14   14
P5         28    21   15   15

Rata-rata  WT=7.40  TAT=13.00  RT=7.40

============================================================
SJF (non-preemptive)
============================================================
Gantt chart:
|    P1    |  P2  |   P4   |     P5     |         P3         |
0          5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=5.40

============================================================
SRTF (SJF preemptive)
============================================================
Gantt chart:
|    P1    |  P2  |   P4   |     P5     |         P3         |
0          5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=5.40

============================================================
Priority (non-preemptive)
============================================================
Gantt chart:
|    P1    |  P2  |   P4   |     P5     |         P3         |
0          5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=5.40

============================================================
Priority (preemptive)
============================================================
Gantt chart:
| P1 |  P2  |  P1  |   P4   |     P5     |         P3         |
0    2      5      8        12           18                   28

Proses     CT   TAT   WT   RT
P1          8     8    3    0
P2          5     3    0    0
P3         28    25   15   15
P4         12     8    4    4
P5         18    11    5    5

Rata-rata  WT=5.40  TAT=11.00  RT=4.80
```

```
============================================================
Round Robin q=4
============================================================
Gantt chart:
|   P1   |  P2  |   P3   |   P4   | P1 |   P5   |   P3   | P5 | P3 |
0        4      7        11       15   16       20       24   26   28

Proses     CT   TAT   WT   RT
P1         16    16   11    0
P2          7     5    2    2
P3         28    25   15    4
P4         15    11    7    7
P5         26    19   13    9

Rata-rata  WT=9.60  TAT=15.20  RT=4.40

============================================================
Round Robin q=6
============================================================
Gantt chart:
|    P1    |  P2  |     P3     |   P4   |     P5     |   P3   |
0          5      8            14       18           24       28

Proses     CT   TAT   WT   RT
P1          5     5    0    0
P2          8     6    3    3
P3         28    25   15    5
P4         18    14   10   10
P5         24    17   11   11

Rata-rata  WT=7.80  TAT=13.40  RT=5.80

============================================================
TABEL PERBANDINGAN
============================================================
Algoritma                     Avg WT  Avg TAT  Avg RT
FCFS                            7.40    13.00    7.40
SJF (non-preemptive)            5.40    11.00    5.40
SRTF (SJF preemptive)           5.40    11.00    5.40
Priority (non-preemptive)       5.40    11.00    5.40
Priority (preemptive)           5.40    11.00    4.80
Round Robin q=4                 9.60    15.20    4.40
Round Robin q=6                 7.80    13.40    5.80

mhs@srv-toko-250660121024:~/k2-simulator$ exit
exit
Script done.
```

## Lampiran B. Lembar Kontribusi Anggota

Persentase kontribusi berjumlah 100% dan disepakati seluruh anggota.

| No | Nama | NIM | Peran dan kontribusi | % | Paraf |
|---|---|---|---|---|---|
| 1 | Diza Asmiranda Subianto | 250660121024 | Koordinator dan penguji simulator | 33,33 |  |
| 2 | Naya Naira Zahra | 250660121067 | Analis dan penghitung manual | 33.33 |  |

| No | Nama | NIM | Peran dan kontribusi | % | Paraf |
|---|---|---|---|---|---|
| 3 | Muhammad Rizky Zamzami | 250660121044 | Penyusun laporan | 33,33 |  |
| | Total | | | 100 | |

## Lampiran C. Log Kerja Kelompok

| Tanggal | Kegiatan | Anggota hadir | Hasil |
|---|---|---|---|
| 15 Oktober 2026 | 1. Pembagian peran dan rencana kerja<br>2. Menyiapkan VM dan memasang toko UMKM<br>3. Menghitung manual 7 skenario<br>4. Membuat dan menjalankan simulator<br>5. Memverifikasi manual vs simulator<br>6. Uji flash sale<br>7. Analisis dan rekomendasi<br>8. Penyusunan laporan<br>9. Pemeriksaan akhir, PDF, dan unggah repositori | Diza Asmiranda S.<br>Naya Naira Z.<br>M.Rizky Zamzami | 1. VM siap, semua pemeriksaan [OK]<br>2. Gantt chart dan tabel manual 7 skenario<br>3. Hasil_simulasi.txt dan tabel perbandingan<br>4. Rata-rata WT, TAT, RT cocok untuk semua skenario<br>5. Uji flash sale 20 berhasil, 80 ditolak, stok akhir 0<br>6. Repositori Github. |

## Lampiran D. Pernyataan Penggunaan AI Generatif

☐ Kelompok kami tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

| Alat | Tujuan penggunaan | Bagian yang dibantu | Cara memverifikasi |
|---|---|---|---|
| Claude (Anthropic) | Memahami konsep dan cara menghitung penjadwalan CPU; membantu menyusun kerangka kode simulator dan draf laporan | Bagian 1, 3, 5, 6 dan Lampiran A | Menghitung ulang Gantt chart secara manual, membandingkannya dengan keluaran simulator (Tabel 6.4), serta menjalankan dan |

| Alat | Tujuan penggunaan | Bagian yang dibantu | Cara memverifikasi |
|---|---|---|---|
| | | | membaca kode sendiri di VM |

# Lampiran E. Daftar Periksa Sebelum Mengumpulkan

- ✓ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.
- ✓ Identitas, kode tugas, dan nama host VM pada sampul sudah benar.
- ✓ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.
- ✓ Setiap tangkapan layar menampilkan prompt/nama host VM dan memiliki nomor serta keterangan.
- ✓ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
- ✓ Lembar kontribusi sudah diisi dan disetujui semua anggota (total 100%).
- ✓ Kode dan konfigurasi sudah diunggah ke repositori kelompok.
- ✓ Pernyataan penggunaan AI generatif sudah diisi.
- ✓ Daftar isi sudah diperbarui (klik kanan → Update Field).
- ✓ Berkas disimpan sebagai PDF dengan nama <Kelas>_<Kode>_Kelompok<N>.pdf.

## Swa-nilai berdasarkan kriteria lembar tugas

Kriteria dan bobot disalin dari lembar tugas K-2.

| Kriteria (dari lembar tugas) | Bobot | Sudah terpenuhi? Di bagian mana? |
|---|---|---|
| Hasil/keberhasilan implementasi | 35% | Ya: Bagian 5, 6.1, 6.2, dan Lampiran A |
| Kedalaman analisis | 30% | Ya: Bagian 6.3 dan 6.4 |
| Dokumentasi | 20% | Ya: seluruh laporan, gambar bernomor, dan lampiran |
| Kerja sama dan pembagian peran | 15% | Ya: Bagian 2, Lampiran B, dan Lampiran C |